フォーマットファイル詳細モーダル UI仕様書

1. 概要

1.1 コンポーネントの目的

フォーマットファイル詳細モーダルは、選択されたフォーマットファイルの詳細情報（メタデータ）を表示するための読み取り専用モーダルダイアログです。ファイル名、サイズ、更新日、作成者など、ファイルに関する包括的な情報を提供します。

1.2 使用場面

- **表示場所**: フォーマット管理画面のモーダルオーバーレイ
- **トリガー**: ファイル行のクリック、または「詳細」ボタンのクリック
- **主な用途**:
  - ファイルの基本情報確認
  - ファイルパスの確認
  - 作成者・作成日の確認
  - バージョン情報の確認

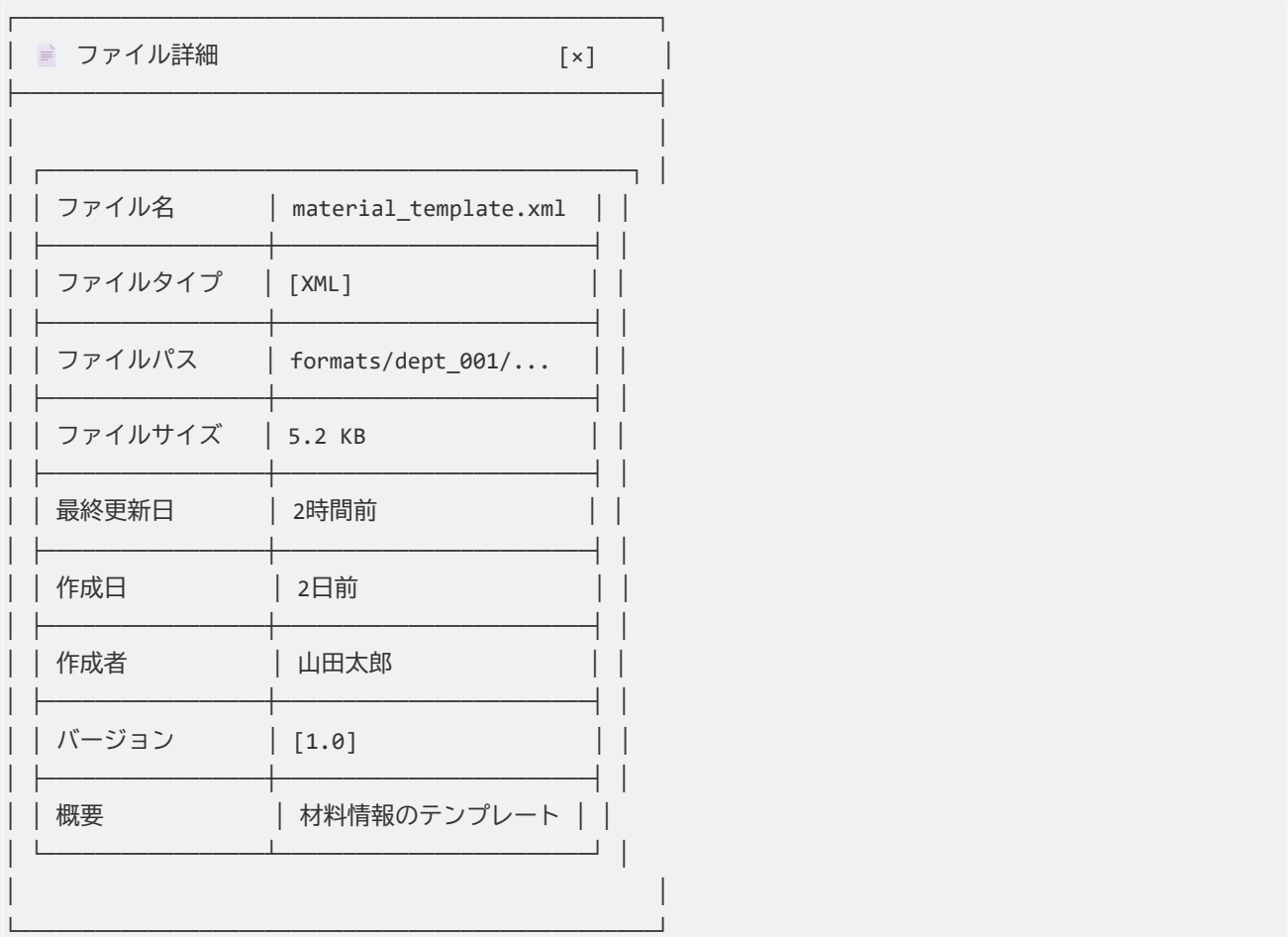
1.3 アクセス権限

- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: 親コンポーネント（FormatManagement）で認証済み

---

2. 画面構成

2.1 モーダル構造



2.2 主要UI要素

2.2.1 モーダルヘッダー

- **アイコン**: FileTextOutlined（青色）
- **タイトル**: 「ファイル詳細」

2.2.2 詳細情報テーブル（Descriptions）

- **スタイル**: ボーダー付き、1カラム、smallサイズ
- **ラベル幅**: 30%
- **ラベルスタイル**: 太字

2.2.3 表示項目

必須項目（常に表示）

1. **ファイル名**: 太字で表示
2. **ファイルタイプ**: Tagコンポーネント（青色）で拡張子を表示
3. **ファイルパス**: コード表示（灰色背景）
4. **ファイルサイズ**: フォーマット済み（例: `5.2 KB`）
5. **最終更新日**: 相対時間形式（例: `2時間前`）
6. **概要**: 改行保持、または「概要が登録されていません」

オプション項目（条件付き表示）

7. **作成日**: 相対時間形式（`file.createdAt`が存在する場合のみ）
8. **作成者**: 作成者名（`file.createdByName`が存在する場合のみ）
9. **バージョン**: Tagコンポーネント（緑色）（`file.version`が存在する場合のみ）

2.2.4 フッター

- **なし**: 読み取り専用のため、フッターボタンなし（`footer={null}`）

2.3 インタラクション

2.3.1 モーダル表示

- **操作**: ファイル行をクリック
- **結果**: モーダルが開き、ファイル詳細が表示される

2.3.2 モーダルを閉じる

- **操作**: ×ボタンをクリック、またはモーダル外側をクリック、またはEscキーを押下
- **結果**: モーダルが閉じる

---

3. 機能仕様

3.1 ファイル詳細情報の表示

3.1.1 Descriptionsレンダリング

```
<Descriptions
  bordered
  column={1}
  size="small"
  style={{ marginTop: token.marginLG }}
>
```

- **bordered**: ボーダー付きテーブル
- **column={1}**: 1カラム（ラベル:値の2列構成）
- **size="small"**: 省スペース表示
- **marginTop**: ヘッダーとの間隔

3.1.2 UI動作

- ユーザー操作: なし（自動表示）
- 画面変化: ファイル情報がテーブル形式で表示される

3.2 ファイルタイプの表示

3.2.1 拡張子抽出

```
const extension = file.name.split('.').pop()?.toUpperCase() || '';
```

- **処理**: ファイル名から最後の `.` 以降を抽出し、大文字に変換
- **例**: `material_template.xml` → `XML`

3.2.2 Tagコンポーネント

```
<Tag color="blue">{extension}</Tag>
```

- **色**: 青（`blue`）
- **内容**: 拡張子の大文字

3.2.3 UI動作

- ユーザー操作: なし
- 画面変化: ファイルタイプが青いタグで表示される

3.3 ファイルパスの表示

3.3.1 コード表示

```
<Text type="secondary" code>{file.path}</Text>
```

- **type="secondary"**: 灰色テキスト
- **code**: コードスタイル（灰色背景、等幅フォント）

3.3.2 UI動作

- ユーザー操作: なし
- 画面変化: ファイルパスがコードブロックとして表示される

3.4 フォーマッター関数の使用

3.4.1 ファイルサイズのフォーマット

```
{formatFileSize(file.size)}
```

- **入力**: バイト数（number）
- **出力**: 人間が読みやすい形式（例: `5.2 KB`, `1.5 MB`）

3.4.2 相対時間のフォーマット

```
{formatRelativeTime(file.lastModified)}
{formatRelativeTime(file.createdAt)}
```

- **入力**: ISO 8601形式の日時文字列
- **出力**: 相対時間（例: `2時間前`, `1日前`, `3週間前`）

3.4.3 UI動作

- ユーザー操作: なし
- 画面変化: フォーマット済みの値が表示される

3.5 条件付き表示

3.5.1 作成日の条件付き表示

```
{file.createdAt && (
  <Descriptions.Item
    label={t('formatManagement.detail.createdAt') || '作成日'}
    labelStyle={{ width: '30%', fontWeight: token.fontWeightStrong }}
  >
    <Text>{formatRelativeTime(file.createdAt)}</Text>
  </Descriptions.Item>
)}
```

- **表示条件**: `file.createdAt` が存在する場合のみ

3.5.2 作成者の条件付き表示

```
{file.createdByName && (
  <Descriptions.Item
    label={t('formatManagement.detail.createdBy') || '作成者'}
    labelStyle={{ width: '30%', fontWeight: token.fontWeightStrong }}
```

```
  >
    <Text strong>{file.createdByName}</Text>
  </Descriptions.Item>
)}
```

- **表示条件**: `file.createdByName` が存在する場合のみ
- **スタイル**: 太字

3.5.3 バージョンの条件付き表示

```
{file.version && (
  <Descriptions.Item
    label={t('formatManagement.detail.version') || 'バージョン'}
    labelStyle={{ width: '30%', fontWeight: token.fontWeightStrong }}
  >
    <Tag color="green">{file.version}</Tag>
  </Descriptions.Item>
)}
```

- **表示条件**: `file.version` が存在する場合のみ
- **スタイル**: 緑色のTag

3.5.4 UI動作

- ユーザー操作: なし
- 画面変化:
  - データが存在する場合: 該当項目が表示される
  - データが存在しない場合: 該当項目は表示されない（テーブル行が省略される）

3.6 概要の表示

3.6.1 概要あり

```
{file.description ? (
  <Text style={{ whiteSpace: 'pre-wrap' }}>{file.description}</Text>
) : (
  <Text type="secondary" italic>
    {t('formatManagement.detail.noDescription') || '概要が登録されていません'}
  </Text>
)}
```

- **スタイル**: `whiteSpace: 'pre-wrap'`（改行を保持）
- **内容**: ファイルの説明文

3.6.2 概要なし

- **スタイル**: 灰色、イタリック
- **内容**: 「概要が登録されていません」

3.6.3 UI動作

- ユーザー操作: なし
- 画面変化:
  - 概要がある場合: 改行を保持して表示
  - 概要がない場合: プレースホルダーテキストを表示

3.7 モーダルを閉じる

3.7.1 閉じる処理

```
onCancel={onClose}
```

- **トリガー**: ×ボタン、モーダル外側クリック、Escキー
- **動作**: `onClose()` コールバックが呼び出される

3.7.2 UI動作

- ユーザー操作: ×ボタンをクリック
- 画面変化: モーダルが閉じる

---

4. データ仕様

4.1 Props定義

```
interface FormatFileDetailModalProps {
  open: boolean;                                    // モーダル表示状態
  onClose: () => void;                              // モーダルを閉じる時のコールバック
  file: FormatFileDetailData | null;                // 表示するファイルデータ
}
```

Props説明:

Prop名	型	必須	説明
open	boolean	✅	モーダルの表示状態（true=表示、false=非表示）
onClose	() => void	✅	モーダルを閉じる時に呼び出されるコールバック関数
file	FormatFileDetailData | null	✅	表示するファイルの詳細データ（null=データなし）

4.2 FormatFileDetailData型定義

```
export interface FormatFileDetailData {
  id: string;                          // ファイルID
  name: string;                        // ファイル名
  path: string;                        // S3/Swiftパス
  size: number;                        // ファイルサイズ（バイト）
  lastModified: string;                // 最終更新日時（ISO 8601形式）
  type: string;                        // MIMEタイプまたはファイルタイプ
  version?: string;                    // バージョン（オプション）
  description?: string;                // 説明（オプション）
  createdBy?: string;                  // 作成者ユーザーID（オプション）
  createdByName?: string;              // 作成者名（オプション）
```

```
  createdAt?: string;                // 作成日時（ISO 8601形式、オプション）
  // 以下は将来的に使用される可能性があるプロパティ
  content?: string;                  // ファイルの内容（編集用）
  isNewFile?: boolean;               // 新規ファイルフラグ
  parsedTemplate?: any;              // パース済みTemplate型オブジェクト
  parsedProperties?: any;            // パース済みPropertyData
  parsedContents?: any;              // パース済みContentData
}
```

FormatFileDetailData説明:

フィールド名	型	必須	説明
id	string	✅	ファイルの一意識別子
name	string	✅	ファイル名（拡張子を含む）
path	string	✅	S3/Swiftの完全パス
size	number	✅	ファイルサイズ（バイト単位）
lastModified	string	✅	最終更新日時（ISO 8601形式文字列）
type	string	✅	MIMEタイプまたはファイルタイプ
version	string	-	ファイルのバージョン番号
description	string	-	ファイルの説明文（改行を含む可能性あり）
createdBy	string	-	作成者のユーザーID
createdByName	string	-	作成者の名前（表示用）
createdAt	string	-	作成日時（ISO 8601形式文字列）

4.3 State定義

このコンポーネントは**ステートレス**で、ローカルStateを持ちません。 全ての情報はPropsとして渡されます。

---

5. API連携

このコンポーネントは直接APIを呼び出しません。 親コンポーネント（FormatManagement）が取得したデータを `file` Propsとして受け取ります。

---

6. エラーハンドリング

6.1 エラーケース

このコンポーネントは表示専用のため、独自のエラーハンドリングはありません。

以下のケースは親コンポーネントで処理されます：

- ファイル詳細取得失敗
- データの不整合（nullやundefined）

6.2 Nullチェック

```
if (!file) {
  return null;
}
```

- **目的**: `file` がnullの場合、何も表示しない
- **タイミング**: レンダリング前

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/project/FormatFileDetailModal.tsx`

7.2 関連ファイル

- **親コンポーネント**: `frontend/src/components/project/FormatManagement.tsx`
- **ユーティリティ**:

    - `frontend/src/utils/formatters.ts` (formatFileSize, formatRelativeTime)

- **コンテキスト**: `frontend/src/contexts/LanguageContext.tsx`
- **型定義**: このファイル内で定義・export（FormatFileDetailData型）

7.3 依存ライブラリ

- **Ant Design**:
    - Modal
    - Descriptions
    - Space
    - Typography (Text)
    - theme
    - Tag
- **Ant Design Icons**:
    - FileTextOutlined
- **Context**:
    - useLanguage（多言語対応）

7.4 親子関係

```
FormatManagement (親)
  ↓ Props: open, onClose, file
FormatFileDetailModal (このコンポーネント)
```

親コンポーネントから受け取るデータ:

- モーダル表示状態
- ファイル詳細データ

親コンポーネントへの通知:

- モーダルを閉じるイベント（`onClose()`）

---

8. UI/UXの考慮事項

8.1 レスポンシブデザイン

- **幅**: 600px（固定）
- **高さ**: コンテンツに応じて可変

8.2 アクセシビリティ

- **キーボード操作**: Escキーでモーダルを閉じる
- **明確な情報**: ラベルが太字で視認性が高い
- **条件付き表示**: データがない項目は表示しない（混乱を避ける）

8.3 視覚的フィードバック

8.3.1 ファイルタイプの識別

- **Tag**: 青色のタグで拡張子を目立たせる

8.3.2 バージョンの識別

- **Tag**: 緑色のタグでバージョンを目立たせる

8.3.3 ファイルパスの識別

- **コードスタイル**: 灰色背景と等幅フォントで技術的な情報を表現

8.3.4 概要の可読性

- **改行保持**: `whiteSpace: 'pre-wrap'` で複数行の説明文を正しく表示

- **プレースホルダー**: 概要がない場合、イタリックで明確に示す

8.4 多言語対応

- **LanguageContext**: `useLanguage` フックを使用
- **翻訳キー**:
  - `formatManagement.detail.title`
  - `formatManagement.detail.fileName`
  - `formatManagement.detail.fileType`
  - `formatManagement.detail.filePath`
  - `formatManagement.detail.fileSize`
  - `formatManagement.detail.lastModified`
  - `formatManagement.detail.createdAt`
  - `formatManagement.detail.createdBy`
  - `formatManagement.detail.version`
  - `formatManagement.detail.description`
  - `formatManagement.detail.noDescription`

8.5 読み取り専用

- **フッターなし**: 編集や保存ボタンは表示しない
- **目的**: 情報の確認のみ（編集は「編集」ボタンから別画面で行う）

---

9. 使用例

9.1 基本的な使用

```
// 親コンポーネントでの呼び出し
<FormatFileDetailModal
  open={isDetailModalOpen}
  onClose={() => setIsDetailModalOpen(false)}
  file={selectedFile}
/>
```

9.2 ファイルデータの例

```
const fileData: FormatFileDetailData = {
  id: 'file_001',
  name: 'material_template.xml',
  path: 'formats/dept_001/material/material_template.xml',
  size: 5324,
  lastModified: '2026-03-11T08:30:00Z',
  type: 'application/xml',
  version: '1.0',
  description: '材料情報のテンプレート\n新規材料登録時に使用',
  createdBy: 'user_001',
  createdByName: '山田太郎',
  createdAt: '2026-03-09T10:00:00Z',
};
```

---

10. 変更履歴

日付	変更内容	担当者
2026-03-11	実装ベースで仕様書を作成（Phase 4: フォーマット管理画面）	Cline